

Faces -- Template (fill in after running experiments)

This file holds the sections that need real numbers, plots, or judgment calls you can only make once `q2q3_run_all_stats.py` has finished.

To populate:

```
python3 q2q3_run_all_stats.py -o results.json
```

or, faces only:

```
python3 q2q3_run_all_stats.py -w perceptron_faces scratch_faces pytorch_faces -o results_faces.json
```

§1 Team and identification

- Team members:

- TODO name 1 -- TODO NetID 1
- TODO name 2 -- TODO NetID 2
- TODO name 3 -- TODO NetID 3

- Submitter: TODO name (NetID)

- Mirror the same names + NetIDs in `team_details.json` (currently still placeholders).
-

§4 Results -- Faces

4.1 Raw results table

Fill from `results.json`. One row per (classifier, training fraction).

Classifier	p (%)	mean error (%)	std error (%)	mean train time (s)
Perceptron	10	TODO	TODO	TODO
Perceptron	20	TODO	TODO	TODO
Perceptron	30	TODO	TODO	TODO
Perceptron	40	TODO	TODO	TODO
Perceptron	50	TODO	TODO	TODO
Perceptron	60	TODO	TODO	TODO
Perceptron	70	TODO	TODO	TODO
Perceptron	80	TODO	TODO	TODO
Perceptron	90	TODO	TODO	TODO
Perceptron	100	TODO	TODO	TODO
Scratch NN	10	TODO	TODO	TODO
Scratch NN	20	TODO	TODO	TODO
Scratch NN	30	TODO	TODO	TODO

Scratch NN	40	TODO	TODO	TODO
Scratch NN	50	TODO	TODO	TODO
Scratch NN	60	TODO	TODO	TODO
Scratch NN	70	TODO	TODO	TODO
Scratch NN	80	TODO	TODO	TODO
Scratch NN	90	TODO	TODO	TODO
Scratch NN	100	TODO	TODO	TODO
PyTorch NN	10	TODO	TODO	TODO
PyTorch NN	20	TODO	TODO	TODO
PyTorch NN	30	TODO	TODO	TODO
PyTorch NN	40	TODO	TODO	TODO
PyTorch NN	50	TODO	TODO	TODO
PyTorch NN	60	TODO	TODO	TODO
PyTorch NN	70	TODO	TODO	TODO
PyTorch NN	80	TODO	TODO	TODO
PyTorch NN	90	TODO	TODO	TODO
PyTorch NN	100	TODO	TODO	TODO

Iterations per cell: TODO (default 5).

4.2 Figure: mean test error vs training fraction (faces)

- x-axis: training fraction (10 -> 100, %).
- y-axis: mean test error (%).
- Three series (Perceptron / Scratch NN / PyTorch NN) with +/- 1 std error bars at every point.

TODO insert figure here. Filename suggestion: `figs/faces\_error.png`.

4.3 Figure: mean training time vs training fraction (faces)

- x-axis: training fraction (10 -> 100, %).
- y-axis: mean training time (s). Switch to log-y if PyTorch swamps the others.
- Three series (Perceptron / Scratch NN / PyTorch NN).

TODO insert figure here. Filename suggestion: `figs/faces\_time.png`.

§5 Discussion -- Faces

5.1 Which algorithm "won"?

- **At p = 100 (full data):** TODO (winner) at TODO % error vs TODO % / TODO % for the other two.
- **At p = 10 (low-data regime):** TODO (which classifier degrades least?).

- **Net call:** TODO -- and explain whether the win is by accuracy, speed, or sample efficiency.

5.2 Accuracy comparison across all three

- Perceptron baseline at full data: TODO %.
- Scratch NN vs Perceptron: TODO percentage points better/worse.
- PyTorch NN vs Scratch NN: TODO percentage points. If they differ, attribute to optimiser (Adam vs SGD), init (Kaiming vs He), or variance, since the architecture is identical.

5.3 Time comparison

- Order from fastest to slowest at $p=100$: TODO -> TODO -> TODO.
- Slope of train time vs p : TODO (roughly linear? sublinear?).
- Cost-per-percentage-point-of-error: TODO (a one-line efficiency take).

5.4 Where did the variance come from?

- Largest std bar observed: TODO classifier at $p = \text{TODO}$ (std = TODO %).
- Source attribution: TODO -- does std shrink toward $p=100$? If yes, subsample randomness dominates; if it stays high, init / shuffle randomness dominates (NNs only).

5.5 Lessons learned

- TODO lesson 1 (e.g. "linear models are surprisingly strong on faces because the task is close to linearly separable in pixel space").
- TODO lesson 2 (e.g. "Adam closes most of the gap to SGD with much less learning-rate tuning").
- TODO lesson 3 (e.g. "for binary tasks of this size, going from a perceptron to a 2-hidden-layer MLP buys X percentage points but costs Y times the training time").

§6 External resources / discussion partners (optional)

- Textbooks / blog posts / tutorials: TODO.
- Discussion partners outside the team: TODO.
- **AI tools used (per README integrity clause):** TODO -- list any LLM assistance and what it was used for (e.g. "ChatGPT / Claude, used for backprop derivation sketch and report wording; all final code and prose reviewed and edited by us").

Pre-submission checklist

- [] `team_details.json` filled with real names + NetIDs.
- [] `q2q3_run_all_stats.py` ran clean end-to-end; `results.json` exists.
- [] Both face figures saved in `figs/` and embedded in the PDF.
- [] Raw results table populated.
- [] Discussion sections (5.1-5.5) all written.
- [] AI / external resources disclosed in §6.
- [] Sanity check from a fresh shell:
 `python3 q1a_perceptron_faces.py 10,`
 `python3 q1b_neural_net_scratch_faces.py 10,`
 `python3 q1c_neural_net_pytorch_faces.py 10 -- all print results.`
- [] `digitdata/` and `facedata/` removed from the zip before upload.
- [] Submission folder named `lastname-netid` and zipped.